

Pixel-Grid LED Earrings

Build a pair of programmable, rechargeable wearable displays — from schematic to PCB to firmware.

DIFFICULTY Intermediate-Advanced

BUILD TIME A weekend + fab lead time

COST ~\$60-120 (small run)

SKILLS Basic EDA, SMD or PCBA service

These earrings hang a tiny 10×10 grid of individually-addressable RGB LEDs from each ear, driven by a fingernail-sized ESP32-C3. They run smooth animations — rainbows, plasma, a pulsing heart — recharge over USB-C, and can later be controlled over Bluetooth. This guide takes you from an empty EasyEDA/KiCad canvas all the way to a finished, wearable pair, including how to order the boards and have them assembled.

01 How it works

The whole design follows one chain: a single LiPo cell feeds the LEDs directly (they tolerate 3.0–4.2 V) and feeds the controller through a small 3.3 V regulator. Running the LEDs straight off the battery removes two parts most beginners add by mistake — a boost converter and a logic level-shifter.

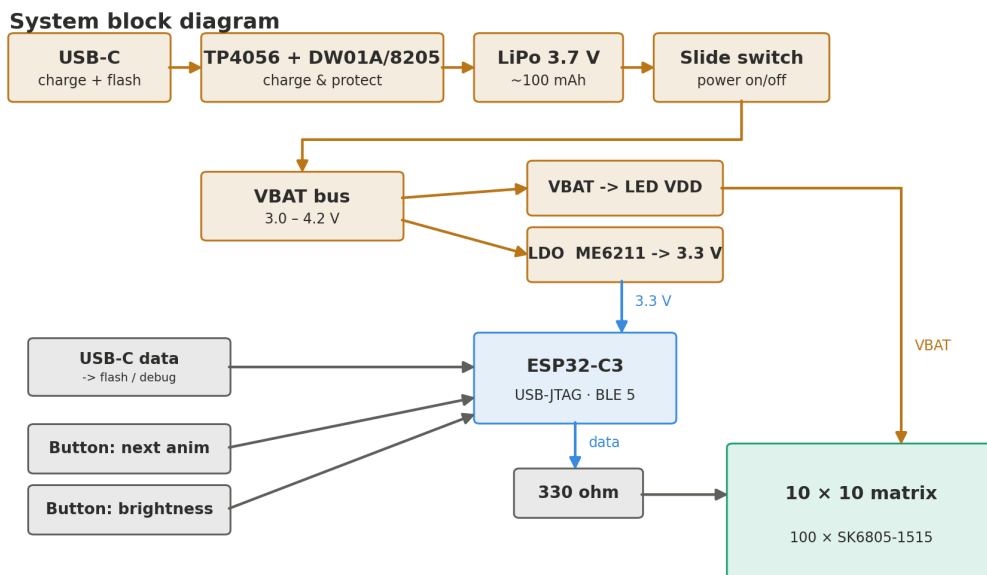


Figure 1 — One earring, end to end. Amber = power, blue = 3.3 V logic, teal = the LED matrix.

Why not WS2812B? Standard WS2812B are 5×5 mm; a 10×10 grid of them is 50×50 mm — the size of a coaster. SK6805-1515 (1.5 mm) shrink the same grid to ~17×17 mm, which is the whole reason this fits on an ear.

02 Bill of materials

Everything below is **per earring** — double it for a pair, and order ~20% spares of the small passives and LEDs (you will lose a few to rework). Confirm each exact part against your fab's in-stock library before ordering.

Part	Qty	Suggested part	Notes
LED	100	SK6805-1515 (or WS2812B-2020)	1.5 mm addressable RGB
MCU	1	ESP32-C3FH4 (QFN32) or ESP32-C3-MINI-1	BLE 5 + USB-Serial-JTAG built in
LDO	1	ME6211C33M5G	3.3 V, ~150 mV dropout
Charger	1	TP4056 (SOP-8)	set charge current via Rprog
Protection	1	DW01A + FS8205A	over-charge / discharge / current
USB	1	USB-C 16-pin receptacle	charge + flash
Switch	1	SPST slide (e.g. SK-12D07)	true power-off
Buttons	1-2	SMD tactile, 3×4 mm	next animation / brightness
Battery	1	1S LiPo ~100 mAh (401230)	the real size constraint
Series R	1	330 Ω 0402/0603	on the LED data line
Bulk cap	1	100 µF	across LED VDD at the matrix
Decoupling	~6	0.1 µF 0402	one per ~10 LEDs + per IC
Resistors	~3	12 kΩ (Rprog), 10 kΩ	charge current + button pull-ups

Earring jewellery You also need the non-electronic bits: surgical-steel ear hooks, a small jump ring, optional clear acrylic/resin front to diffuse the pixels, and a thin enamelled wire or FPC tail if you split the controller onto a second board.

03 Tools & software

- **EDA:** EasyEDA (easiest path to JLCPCB) or KiCad 8 (free, more powerful).
- **Firmware:** VS Code + PlatformIO, or the Arduino IDE 2.x, with the FastLED library.
- **Assembly:** a fine-tip soldering iron + hot-air station, OR just use the fab's assembly (PCBA) service and solder nothing.
- **Bench:** a multimeter, ideally a USB power meter or bench supply with current limit for the first power-on.
- **Tweezers, flux, solder paste,** and a stencil if you reflow by hand.

04 Design the schematic

1 Capture the four blocks

Draw the circuit in the same four groups as Figure 1: charge/protect, power rails, controller, and the LED matrix. Keep each block on its own area of the sheet and label your nets (VBAT, +3V3, LED_DIN, GND) so the layout stage is readable.

2 Wire the key connections

USB-C VBUS → TP4056 IN+ ; TP4056 BAT+ → DW01A/FS8205A → LiPo+
Protected B+ → slide switch → VBAT rail
VBAT → ME6211 VIN ; ME6211 VOUT (3.3 V) → ESP32-C3 3V3 (1μF in & out)
VBAT → LED matrix VDD (100μF bulk cap right at the matrix)
ESP32-C3 GPIO2 → 330Ω → matrix DIN
GPIO3 → NEXT button → GND ; GPIO4 → MODE button → GND
USB-C D+/D- → ESP32-C3 GPIO19/18 (native USB for flashing)

Set the charge current TP4056 charge current is $I_{\text{prog}} = 1200 / R_{\text{prog}}$ (amps, ohms). For a tiny ~100 mAh cell aim for ~1C: a 12 kΩ R_{prog} gives ~100 mA and a ~1.2 h charge. Never fast-charge a cell this small.

05 Lay out the matrix board

The LED matrix is the visible face of the earring, so lay it out first and let everything else serve it. Place the 100 LEDs on a tight grid (1.6–1.8 mm pitch) and chain DIN→DOUT in a serpentine (boustrophedon) path so adjacent rows simply connect end-to-end.

Serpentine pixel order (DIN at bottom-left)

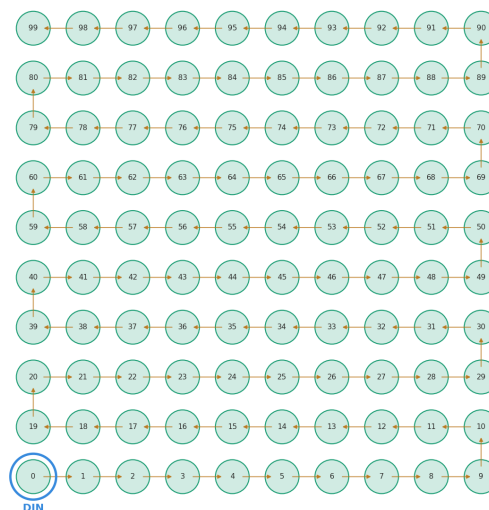


Figure 2 — Serpentine chain order. The number on each pixel is its index in the firmware. Match the SERPENTINE/FLIP_VERTICAL flags in config.h to however you route it.

The firmware already handles this The XY() helper in the firmware converts (x, y) coordinates to this snaking index, so every animation is matrix-agnostic. You only have to make the physical chain order match the two flags in config.h — if an effect runs sideways or upside-down, flip one of them.

06 Lay out the controller

- Put the ESP32-C3 module/chip, LDO, TP4056 and protection on the **back** of the matrix board, or on a small second board joined by a flexible tail — whichever keeps the front all-pixels.
- Give the ESP32-C3 antenna a **keep-out**: no copper pour under or beside the antenna end of the module.
- Keep the 330Ω series resistor and the 100μF bulk cap physically close to the matrix DIN and VDD.
- Place the USB-C connector at the bottom edge so the charging/flashing cable hangs naturally.
- Add a tiny **BOOT pad** (GPIO9 to GND) you can bridge if the very first flash won't enter download mode.

07 PCB layout tips & tricks

Decoupling Scatter 0.1μF caps along the LED chain (about one per 10 pixels) and one beside each IC. Addressable LEDs draw sharp current spikes; local caps stop the data line from glitching.

One solid ground Use a continuous ground pour on the bottom layer and tie the matrix GND back to the battery negative at a single star point. Most 'random flicker' bugs are really ground-bounce.

Protect the data line The 330Ω series resistor at the MCU end damps reflections. Keep DIN short and direct; don't run it as a long thin antenna across the board.

Let FastLED cap the current setMaxPowerInVoltsAndMilliamps() in the firmware auto-dims each frame so the panel can never pull more than your budget — this is your best defence against a full-white frame browning out the regulator. Design the copper for that capped current, not the 1.5 A theoretical max.

Width & finish Make VBAT/GND traces generous (they can carry the LED current), use ENIG finish for fine-pitch reliability, and a 2-layer 0.8 mm board to keep the earring light.

08 Generate fab files & order

1 Export the manufacturing files

From EasyEDA or KiCad export the **Gerbers** (copper, mask, silk, outline), the **BOM** (CSV) and the **CPL / pick-and-place** file (component positions). JLCPCB and PCBWay both want this trio for assembly.

2 Upload and set board options

- 2-layer, 0.8 mm thickness, ENIG finish, your smallest fab-supported trace/space.
- Order a **stencil** if you'll reflow yourself; skip it if you use their PCBA service.
- Check minimum drill/annular-ring rules pass design-rule-check before upload.

3 Pick parts for assembly (PCBA)

- Prefer the fab's **Basic/Preferred** parts — they cost less and skip a loading fee.
- The ESP32-C3 module and SK6805 are commonly stocked; match your footprints to their exact part numbers.
- Hand-place only what they can't: the battery, switch, ear-hook hardware.

Panelize to save money Tiny boards are cheap, but assembly has a per-order setup cost. Panelize several earrings (and a spare) into one array with mouse-bites or V-cuts so one PCBA run yields multiple pairs.

Order spares Always order at least 5 boards (the minimum is usually cheap) and extra LEDs/passives. Your first revision will have a mistake — everyone's does.

09 Assemble & first power-on

- If self-assembling: stencil paste, place parts, reflow on a hotplate/oven, then hand-solder the USB-C, switch and battery.
- Inspect under magnification for bridges on the QFN and the LED pads before applying any power.
- **Smoke test:** power from a current-limited supply (set ~150 mA) *before* connecting the LiPo. Confirm 3.3 V at the ESP32 and no part gets hot.
- Only then connect the battery and verify the TP4056 charges it (charge LED behaviour + rising cell voltage).

LiPo safety Never charge a swollen or damaged cell, never short the pack, and keep the protection IC in the design — not optional on a wearable. Tape/insulate the cell so a sharp edge can't pierce it inside the earring.

10 Flash the firmware

- Install the ESP32 board package and the FastLED library (PlatformIO: `lib_deps = fastled/FastLED`).
- Board = ESP32-C3; enable **USB CDC On Boot** so the native USB serial works.
- Plug in USB-C — the C3 enumerates directly, no FTDI/CH340 adapter needed — pick the port and Upload.
- If the first upload won't catch, bridge the BOOT pad (GPIO9→GND) while plugging in.
- To build the *second* earring or a different size, change `MATRIX_WIDTH/HEIGHT` in `config.h` and re-upload — nothing else.

How many animations? The ESP32-C3 has 4 MB flash and 400 KB RAM; your 100-pixel frame buffer is only 300 bytes and each effect compiles to ~0.3–2 KB. You can ship 50–200+ animations and still use under 5% of flash — the real limit is how many you feel like writing.

11 Power, runtime & safety

Runtime (h) = capacity(mAh) × 0.8 ÷ average current(mA). The honest constraint is weight, not cost — an earring can only carry a tiny cell, so brightness is your main runtime lever.

Battery	Brightness	Avg current	Runtime
100 mAh (401230)	low (~40)	~70 mA	~1.1 h
100 mAh	very low (~20)	~45 mA	~1.8 h
250 mAh (heavier)	low	~90 mA	~2.2 h

12 Make it wearable

- 3D-print or laser-cut a thin frame/back-shell; a frosted acrylic or thin resin layer over the front diffuses the pixels beautifully.
- Mount the LiPo and switch on the back shell, route the ear-hook through a reinforced hole or jump ring.
- Strain-relieve every wire — the USB and battery joints are what fail first on a moving wearable.
- Seal exposed metal so it can't short against hair or skin; keep total weight low for all-day comfort.

13 Troubleshooting

Symptom	Likely cause & fix
Colours swapped (red shows green)	Wrong color order — change GRB to RGB in <code>config.h</code> .
Animation runs sideways / upside-down	Set <code>SERPENTINE / FLIP_VERTICAL</code> to match your wiring.

Symptom	Likely cause & fix
First pixel works, rest are dark/garbled	Cold joint on a DOUT→DIN hop, or missing decoupling caps.
Random flicker, worse when bright	Ground bounce / inrush — add bulk cap at matrix, lower PSU_MILLIAMPS.
Won't flash / no serial port	Enable USB CDC On Boot; bridge BOOT pad on first upload.
Dies after a few minutes	Battery too small or brightness too high — cap brightness in firmware.
Won't charge	Check Rprog value, TP4056 orientation, and protection-IC wiring.

Built with an ESP32-C3 + FastLED. Firmware (config.h, animations.h, LEDEarring.ino) is modular and matrix-agnostic — change two numbers to retarget any panel size. Happy building.